

Caching: Types, Use Cases, and the Problems It Solves

A Practical Overview for Software Engineers

Almost every performance-sensitive system, from a single web server to a globally distributed platform, relies on **caching** in one form or another. Caching means temporarily storing a copy of data that is expensive or slow to produce, so future requests for the same data can be served faster. This report explains what caching is, the main types of caches used in modern software systems, common caching strategies, and the specific problems caching is designed to solve, along with the new problems it can introduce if used carelessly.

Contents

1. What Is Caching and What Problems Does It Solve
2. Types of Caching by Location
3. Caching Strategies and Patterns
4. Cache Eviction and Expiration Policies
5. Common Use Cases
6. Problems Caching Can Introduce
7. Best Practices
8. Conclusion

Scope of this report: caching concepts are largely language- and framework-agnostic, so this overview covers general-purpose caching principles that apply across web applications, APIs, databases, and distributed systems, with concrete examples from common technologies such as Redis, HTTP caching headers, and in-memory application caches.

1. What Is Caching and What Problems Does It Solve

A cache is a storage layer that keeps a copy of data closer to where it is needed, or in a faster medium than its original source, so subsequent reads avoid repeating expensive work. The original, authoritative data still lives somewhere else, a database, a remote API, or the result of a heavy computation, and the cache simply holds a temporary, faster-to-access copy of it.

1.1 The Core Problem: Latency and Load

Without caching, every request for a piece of data has to go all the way back to its original source. If that source is a disk-based database, a third-party API, or a computation that takes seconds, every single request pays that full cost again, even when the data has not changed at all since the last request. As traffic grows, this repeated cost multiplies, and the origin system becomes a bottleneck, and often the first thing to fail under load.

1.2 Specific Problems Caching Addresses

- **High latency** — repeatedly fetching data from a slow source (disk, network, remote service) adds up; a cache serves the same data from a much faster medium, often memory.
- **Redundant computation** — expensive calculations (aggregations, rendering, machine learning inference) do not need to be repeated if the inputs have not changed.
- **Database and backend overload** — every uncached read hits the database directly; under high traffic this can exhaust connections or degrade performance for everyone.
- **Third-party API rate limits and cost** — external APIs often charge per call or throttle requests; caching responses reduces the number of calls made.
- **Scalability limits** — a single database or service instance can only handle so much load; caching allows a system to serve far more requests with the same backend capacity.
- **Poor user experience under load** — slow responses during traffic spikes directly translate into slow pages and timeouts for end users.

2. Types of Caching by Location

Caches can exist at almost every layer of a system's architecture. Each layer trades off speed, scope, and complexity differently.

Cache Type	Where It Lives	Typical Use
Client-side / Browser cache	The user's device/browser	Cached static assets, HTTP responses via Cache-Control headers
CDN cache	Edge servers close to users	Static files, images, videos, sometimes full HTML pages
In-memory application cache	Inside the application process (e.g. a dictionary, MemoryCache)	Frequently accessed data within a single server instance
Distributed cache	A separate service shared by many app instances (e.g. Redis, Memcached)	Session data, shared computed results across a cluster
Database query cache	Inside or in front of the database	Repeated query results, avoiding re-execution of expensive queries
DNS cache	Resolver / OS level	Recently resolved domain-to-IP mappings

Table 1. Common cache layers across a typical system.

2.1 In-Memory vs. Distributed Caching

In-memory caches (such as a local dictionary or a library like MemoryCache in .NET) are extremely fast because there is no network hop, but they are local to a single process; if an application runs on multiple servers, each one holds its own separate copy, which can lead to inconsistent data across instances. Distributed caches such as Redis or Memcached run as a separate shared service, so every application instance reads and writes the same cached data, at the cost of a small network round trip.

3. Caching Strategies and Patterns

Beyond deciding where a cache lives, systems must decide how data flows into and out of the cache. The following patterns are the most widely used.

3.1 Cache-Aside (Lazy Loading)

The application checks the cache first. On a miss, it reads from the original source, stores the result in the cache, and returns it. This is the most common pattern because the cache only ever holds data that has actually been requested.

```
var data = cache.Get(key);
if (data == null)
{
    data = database.Query(key);
    cache.Set(key, data, expiration: TimeSpan.FromMinutes(10));
}
return data;
```

3.2 Read-Through

Similar to cache-aside, but the cache itself is responsible for loading missing data from the source, so the application only ever talks to the cache, never directly to the database.

3.3 Write-Through

Every write goes to the cache and the underlying data store together, in the same operation, keeping the cache always up to date at the cost of slightly slower writes.

3.4 Write-Behind (Write-Back)

Writes go to the cache immediately and are persisted to the underlying store asynchronously afterward. This improves write performance but introduces a window where the durable store is behind the cache, risking data loss if the cache fails before the write is flushed.

3.5 Refresh-Ahead

The cache proactively refreshes frequently accessed entries shortly before they expire, so users rarely experience a slow cache miss for hot data.

4. Cache Eviction and Expiration Policies

Because cache storage is limited, and because cached data can become stale, every cache needs rules for when to remove or refresh entries.

- **TTL (Time to Live)** — each entry expires automatically after a fixed duration, bounding how stale data can become.
- **LRU (Least Recently Used)** — when the cache is full, the entry that has not been accessed for the longest time is removed first.
- **LFU (Least Frequently Used)** — evicts the entry accessed the fewest number of times, favoring consistently popular data.
- **Explicit invalidation** — the application actively removes or updates a cache entry when the underlying data changes, keeping the cache accurate at the cost of extra invalidation logic.

5. Common Use Cases

- **Web page and API response caching** — storing rendered pages or JSON responses so identical requests are served instantly without recomputation.
- **Database query result caching** — avoiding repeated execution of expensive or frequent queries, especially read-heavy, rarely-changing data such as product catalogs.
- **Session storage** — keeping user session data in a fast, shared store (often Redis) so any server in a cluster can serve a logged-in user.
- **Static asset delivery via CDN** — images, scripts, and stylesheets cached at edge locations close to users worldwide, reducing load time and origin server traffic.
- **Rate limiting and counters** — using fast in-memory or distributed caches to track request counts per user or API key.
- **Computation memoization** — caching the result of expensive pure functions (reports, aggregations, machine learning predictions) keyed by their input parameters.
- **DNS and configuration caching** — avoiding repeated network lookups for values that rarely change.

6. Problems Caching Can Introduce

Caching trades one set of problems for another. Understanding these trade-offs is essential to using caching correctly.

- **Stale data** — the cache can serve outdated information if the underlying data changes and the cache is not properly invalidated or refreshed.
- **Cache invalidation complexity** — famously one of the hardest problems in computer science; deciding exactly when and what to invalidate can become intricate in systems with many interrelated entities.
- **Cache stampede (thundering herd)** — when a popular cache entry expires, many simultaneous requests can hit the origin at once, temporarily overwhelming it.
- **Increased architectural complexity** — an additional moving part (the cache) must be deployed, monitored, and kept available.
- **Memory and cost overhead** — caches consume memory or a paid managed service, and sizing them correctly requires ongoing tuning.
- **Consistency issues in distributed systems** — multiple cache nodes or application instances may briefly disagree on the current value of a piece of data.

7. Best Practices

- **Cache what is expensive and frequently read**, not everything; caching rarely-accessed or cheap-to-compute data wastes memory for little benefit.
- **Always set a TTL**, even a generous one, as a safety net against permanently stale data if invalidation logic has a bug.
- **Use a distributed cache for multi-instance applications** to avoid inconsistent results between servers.
- **Guard against cache stampedes** using techniques such as locking, request coalescing, or staggered expiration times.
- **Version or namespace cache keys** so that deploying a schema change does not return old, incompatible cached objects.
- **Monitor hit/miss ratios** to verify the cache is actually reducing load rather than sitting unused.

8. Conclusion

Caching is one of the most effective tools for improving performance and scalability, precisely because it targets the most common source of slowness in software systems: repeating expensive work that has already been done. From browser caches and CDNs at the edge, to in-memory and distributed caches inside the backend, each layer reduces latency, lowers load on origin systems, and helps applications scale to far more users than the underlying data source could handle alone. The trade-off is added complexity, chiefly around keeping cached data fresh and handling the edge cases that arise in distributed environments. Used deliberately, with clear invalidation rules and sensible expiration policies, caching remains one of the highest-leverage techniques available for building fast, resilient systems.

End of report.